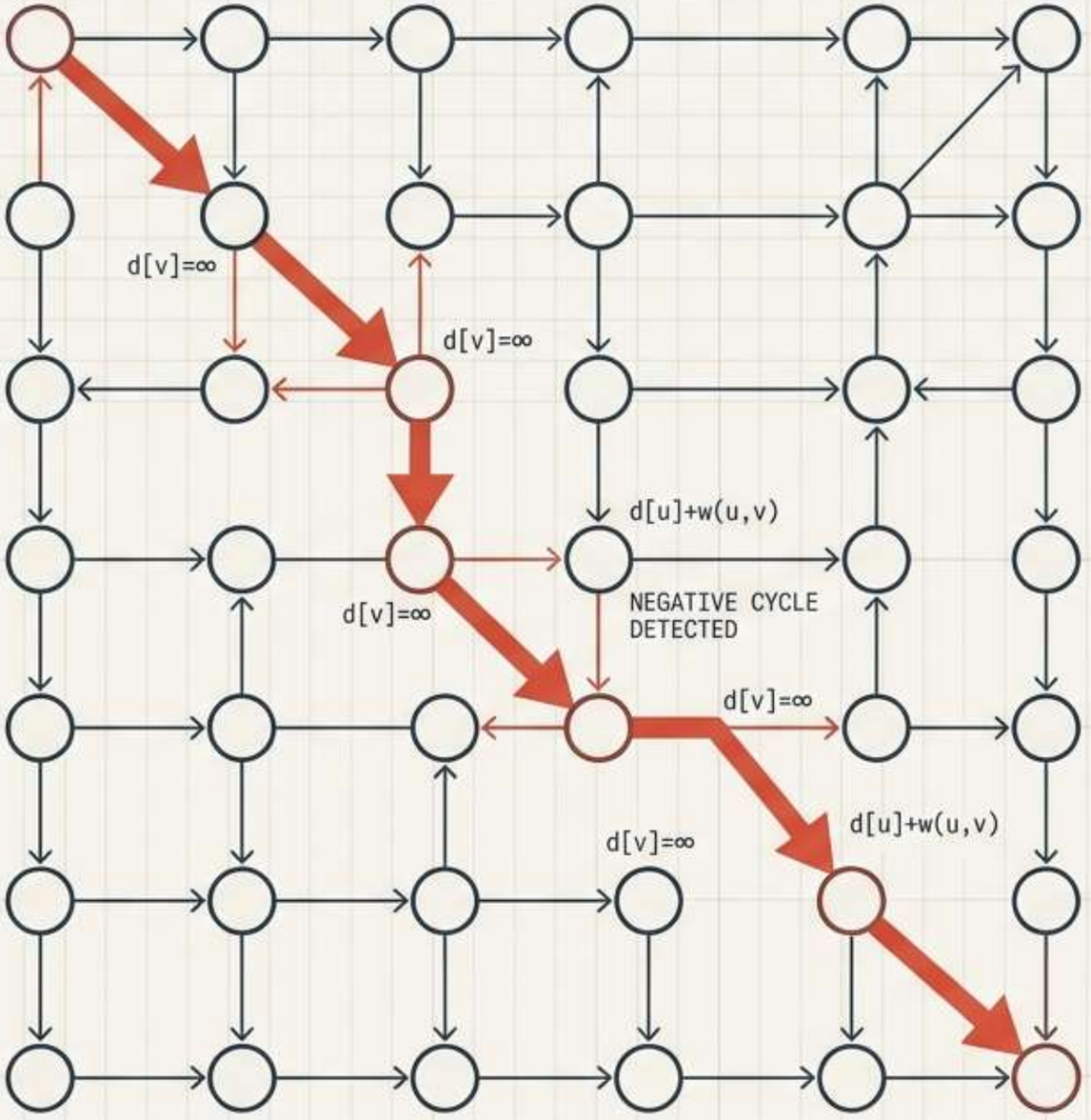
THE BELLMAN-FORD ALGORITHM

MASTERING SHORTEST PATHS, NEGATIVE WEIGHTS, AND ARBITRAGE IN GRAPH THEORY

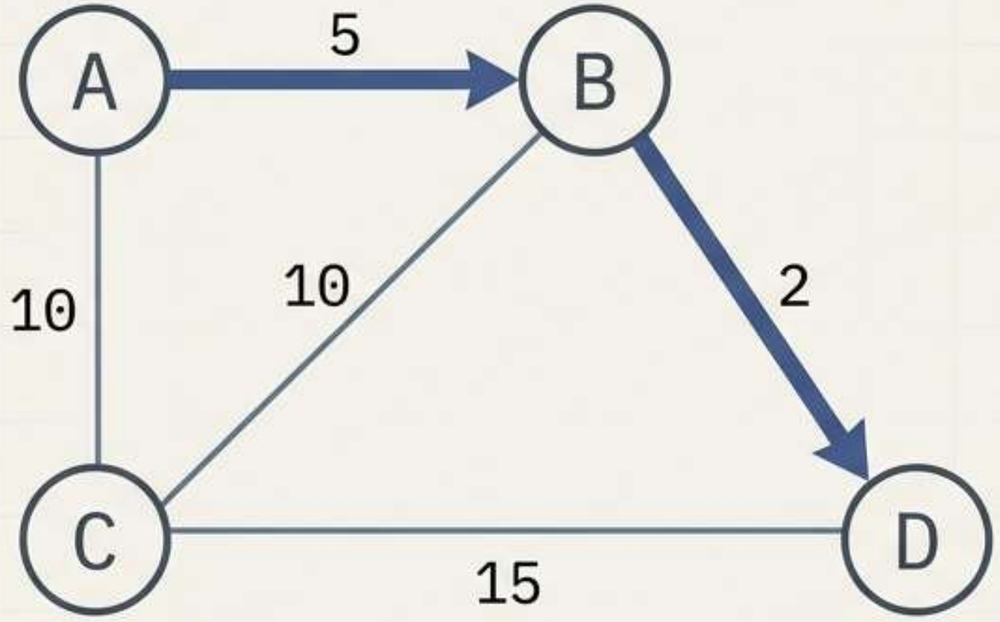


Gemini Notebook

The Achilles Heel of Greedy Pathfinding

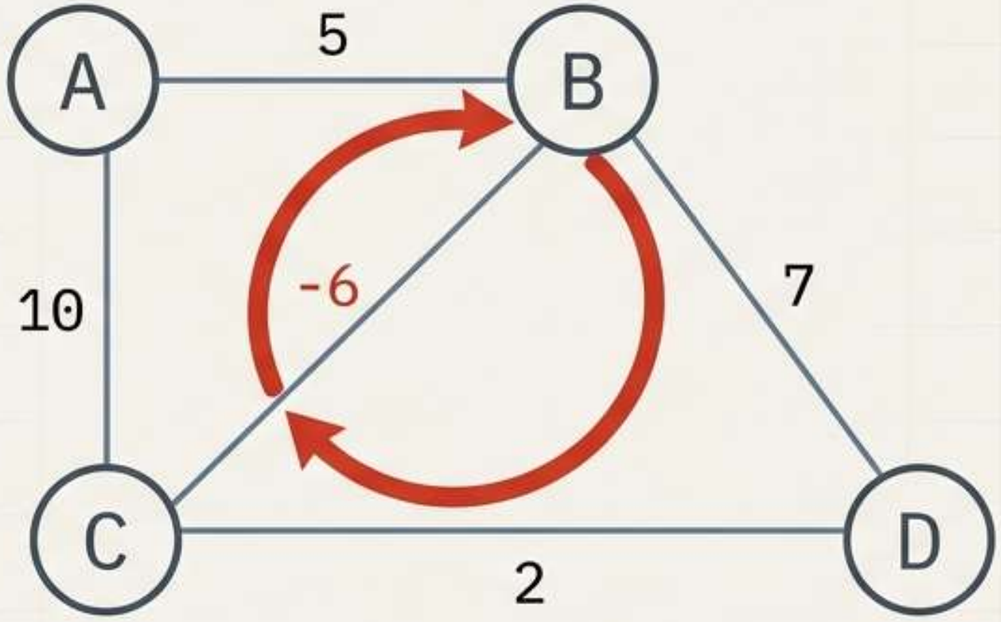
Dijkstra's Algorithm is exceptionally fast, but its greedy paradigm fails catastrophically when introduced to Negative Edge Weights.

Dijkstra on Positive Weights



Assumption holds: Paths strictly get longer as you travel.

Dijkstra on Negative Weights



Fatal flaw: The algorithm gets permanently trapped, continuously discovering "shorter" paths by traveling in circles.

Gemini Notebook

Dynamic Programming to the Rescue

The Bellman-Ford algorithm trades raw speed for absolute resilience. As a Single-Source Shortest Path (SSSP) algorithm, it abandons the greedy approach in favor of Dynamic Programming.

Paradigm	Target Topologies
Dynamic Programming Systematically computes solutions bottom-up.	**Directed & Undirected Graphs** Applicable to varied network structures.
Handles Negative Weights **TRUE** Successfully calculates accurate distances despite cost reductions.	**Detects Impossible Networks** **TRUE** Identifies infinite negative loops and safely aborts.

Gemini Notebook

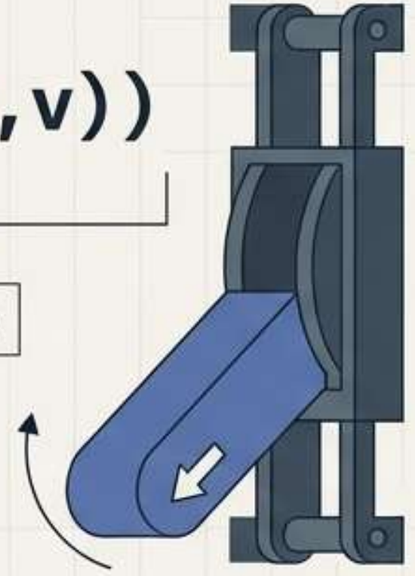
Architectural Transit Blueprint

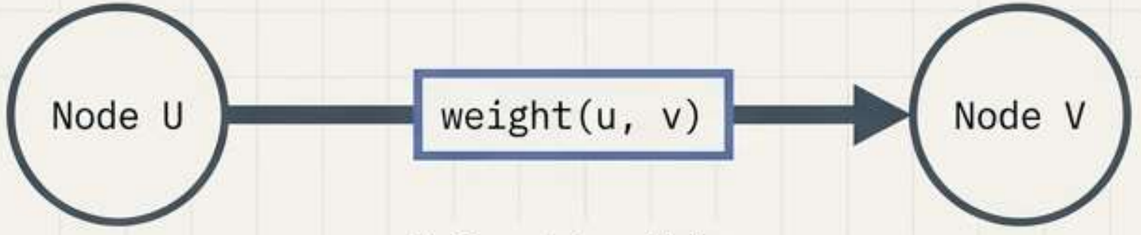
The Core Mechanic: Edge Relaxation

“Relaxation” is the heartbeat of the algorithm. It asks a simple question: If traveling through intermediate node U makes the trip to destination node V cheaper, update the master record to use the new route.

dist[v] = min(dist[v], dist[u] + weight(u,v))

- dist[v] — Updated Best Route
- dist[v] — Current Best
- dist[u] + weight(u,v) — Cost of the Proposed New Route



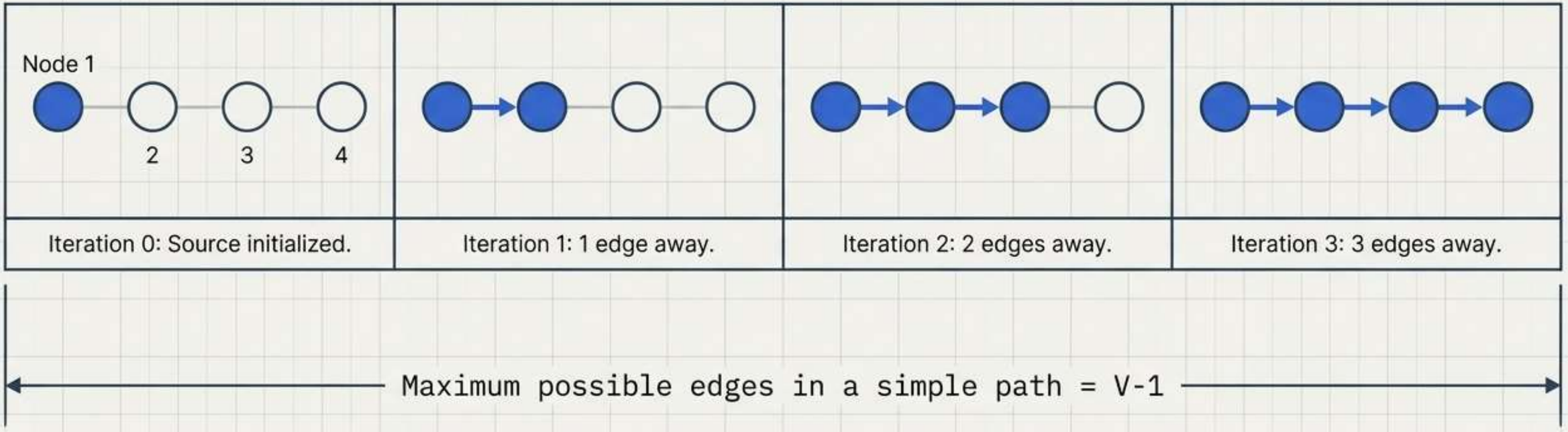


Relaxation Valve

Gemini Notebook

The Timeline of Truth: Why V-1 Iterations?

A valid shortest path in a graph with V vertices can contain, at most, V-1 edges. By relaxing every single edge exactly V-1 times, Bellman-Ford guarantees that the optimal path has enough time to propagate from the source to the furthest possible node.

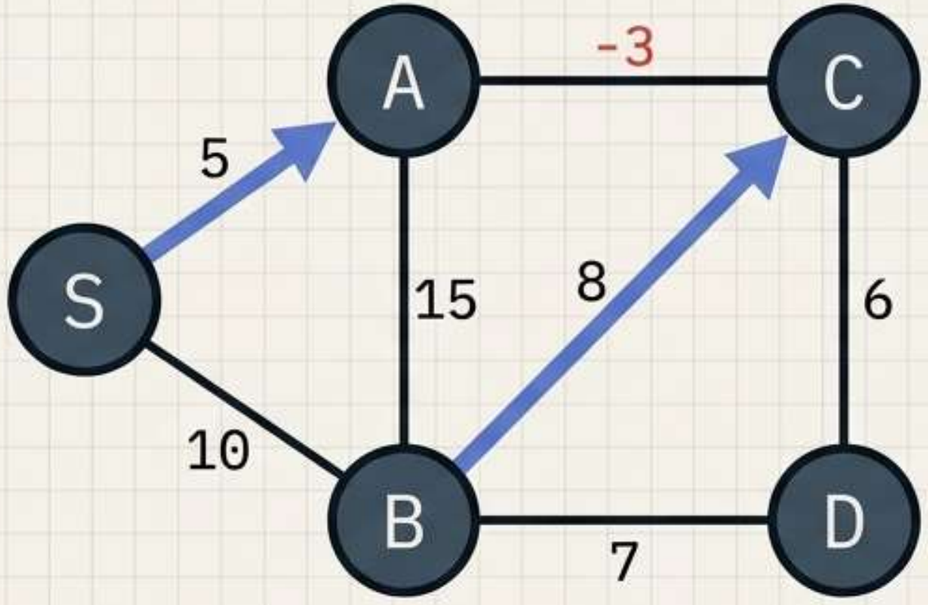


Gemini Notebook

Phase 1: Propagating the Shortest Paths

The algorithm initializes all distances to ∞ and sweeps through every edge V-1 times, applying the relaxation formula.

[STATUS: Iteration 2 of V-1]



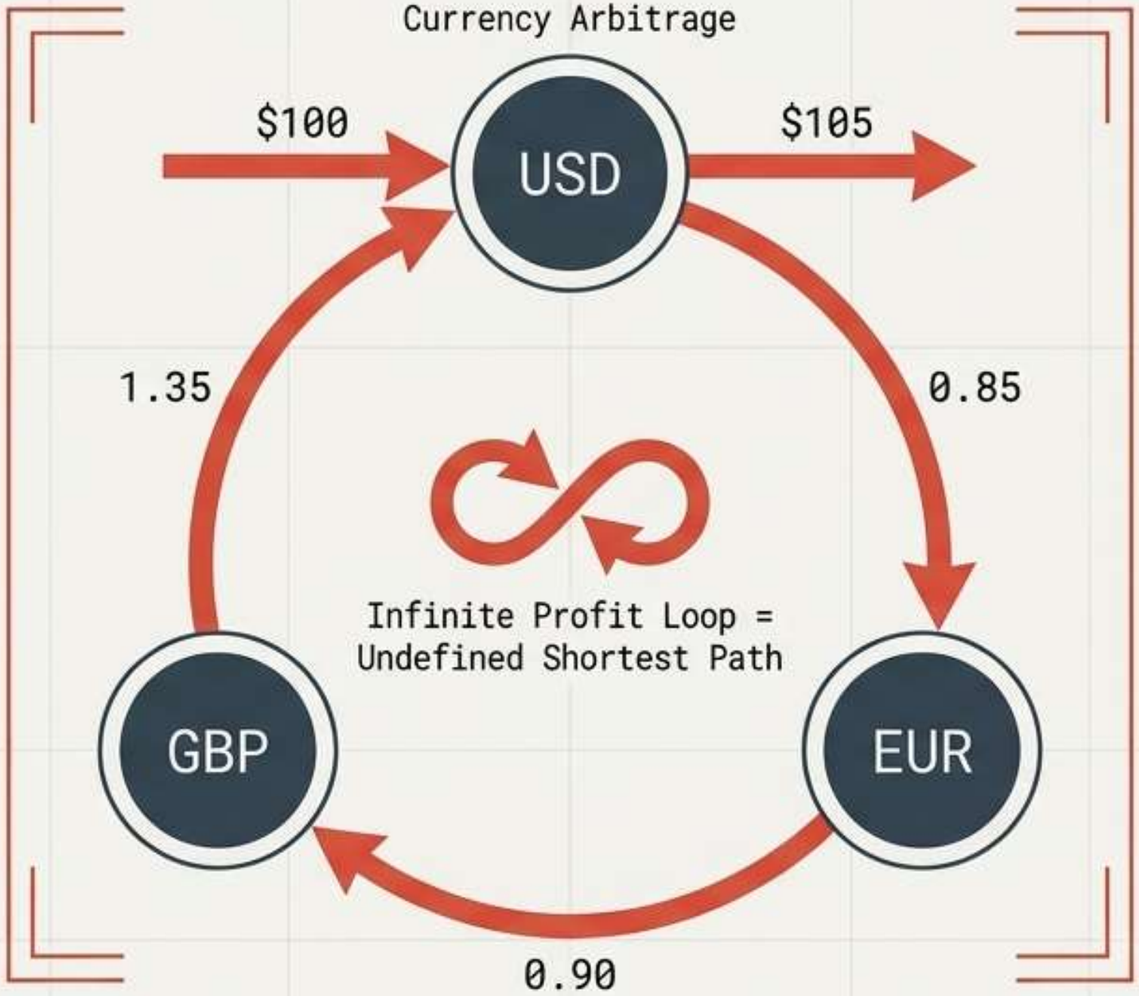
Vertex	Distance
S	0
A	~~∞~~ 5
B	~~∞~~ 10
C	~~∞~~ 18
D	~~∞~~ 17

CRUCIAL DETAIL: Edges can be processed in ANY arbitrary order. The V-1 iteration limit mathematically guarantees the correct shortest paths will eventually resolve regardless of sequence.

Gemini Notebook

The Menace of the Negative Weight Cycle

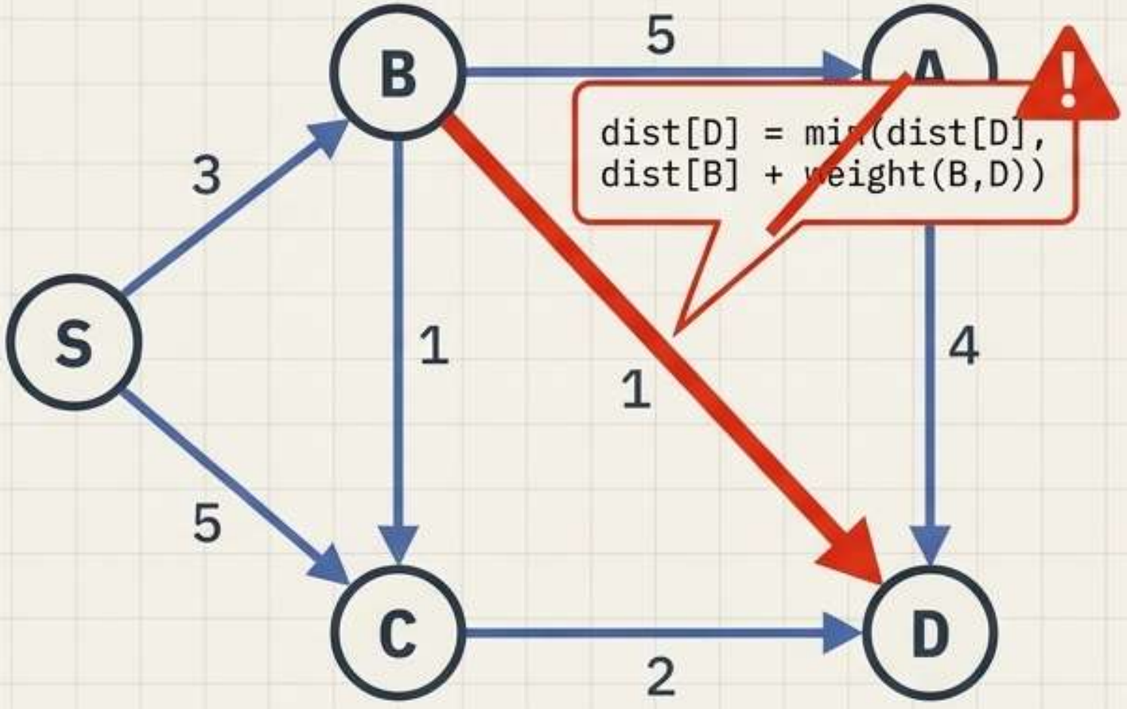
A negative weight cycle is a loop where the sum of edge weights is less than zero. When this occurs, the concept of a "shortest path" becomes mathematically undefined, as an algorithm could traverse the cycle infinitely to drive the cost down to -∞.



Gemini Notebook

Phase 2: The V-th Iteration

After V−1 iterations, all legitimate shortest paths are guaranteed to be found. Bellman-Ford runs exactly one more full pass. If any edge can still be successfully relaxed, it proves the graph is constantly shrinking—confirming a negative weight cycle.

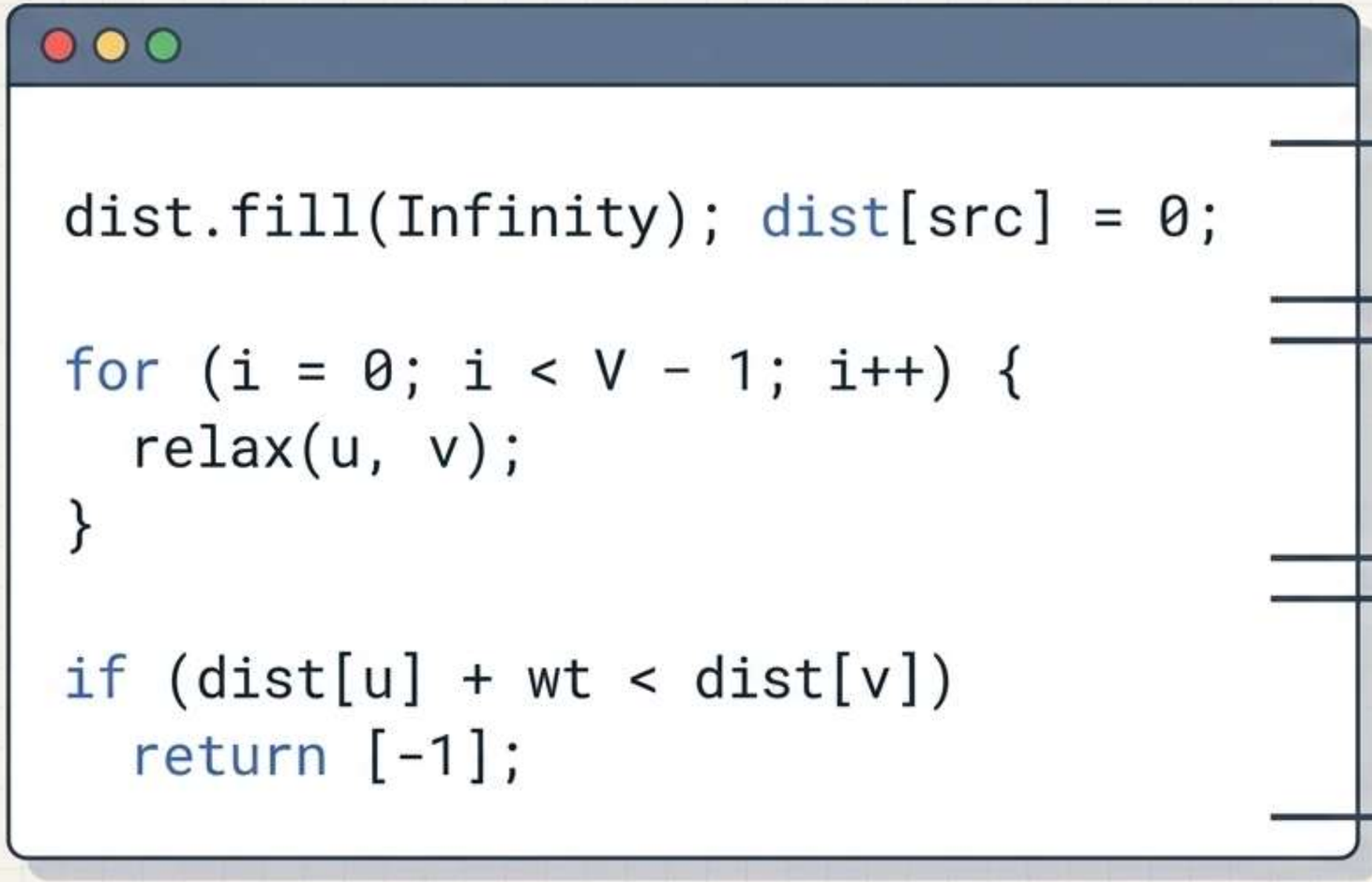


SYSTEM WARNING: CYCLE DETECTED

Relaxation successful on iteration V. Aborting pathfinding. Returning failure flag.

Gemini Notebook

Anatomy of the Algorithm



```
dist.fill(Infinity); dist[src] = 0;

for (i = 0; i < V - 1; i++) {
  relax(u, v);
}

if (dist[u] + wt < dist[v])
  return [-1];
```

Initialize baseline

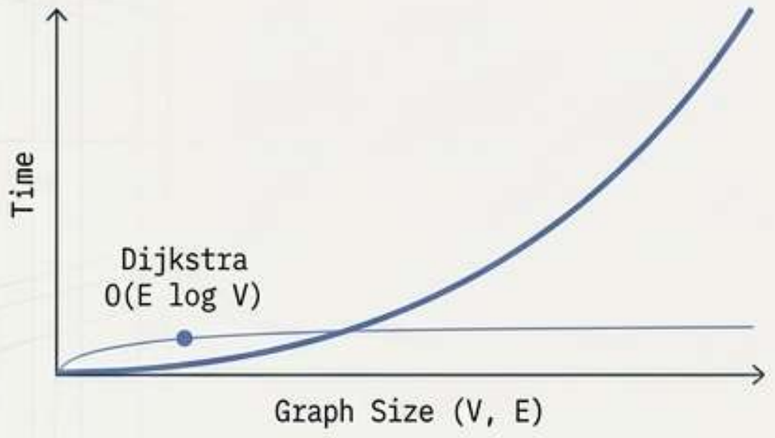
Phase 1:
Propagate Paths

Phase 2:
Detect Cycles

Gemini Notebook

Performance Profiling: The Cost of Resilience

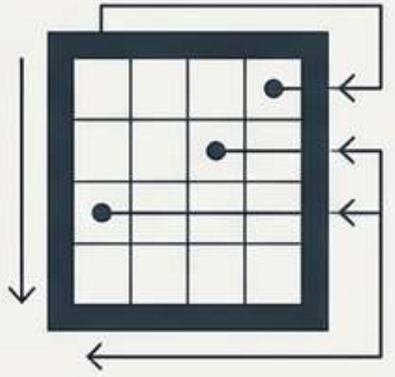
$O(V \times E)$



Because it evaluates every edge (E) across every vertex minus one (V-1), Bellman-Ford is significantly slower than Dijkstra.

In dense graphs where $E \approx V^2$, it approaches $O(V^3)$.

$O(V)$



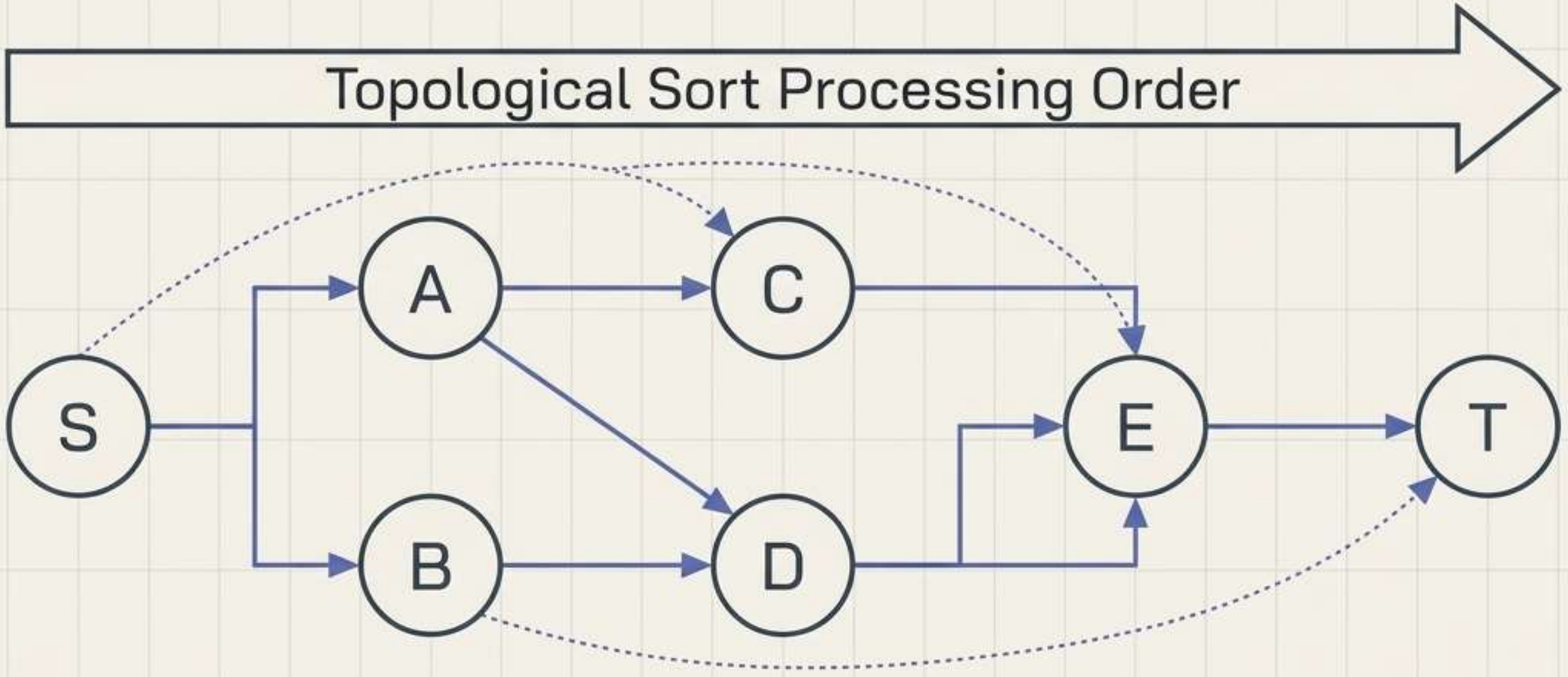
Highly efficient memory footprint.

It requires only a single **1D array** to store the tracked distances from the source node.

Gemini Notebook

The DAG Exception: A High-Speed Shortcut

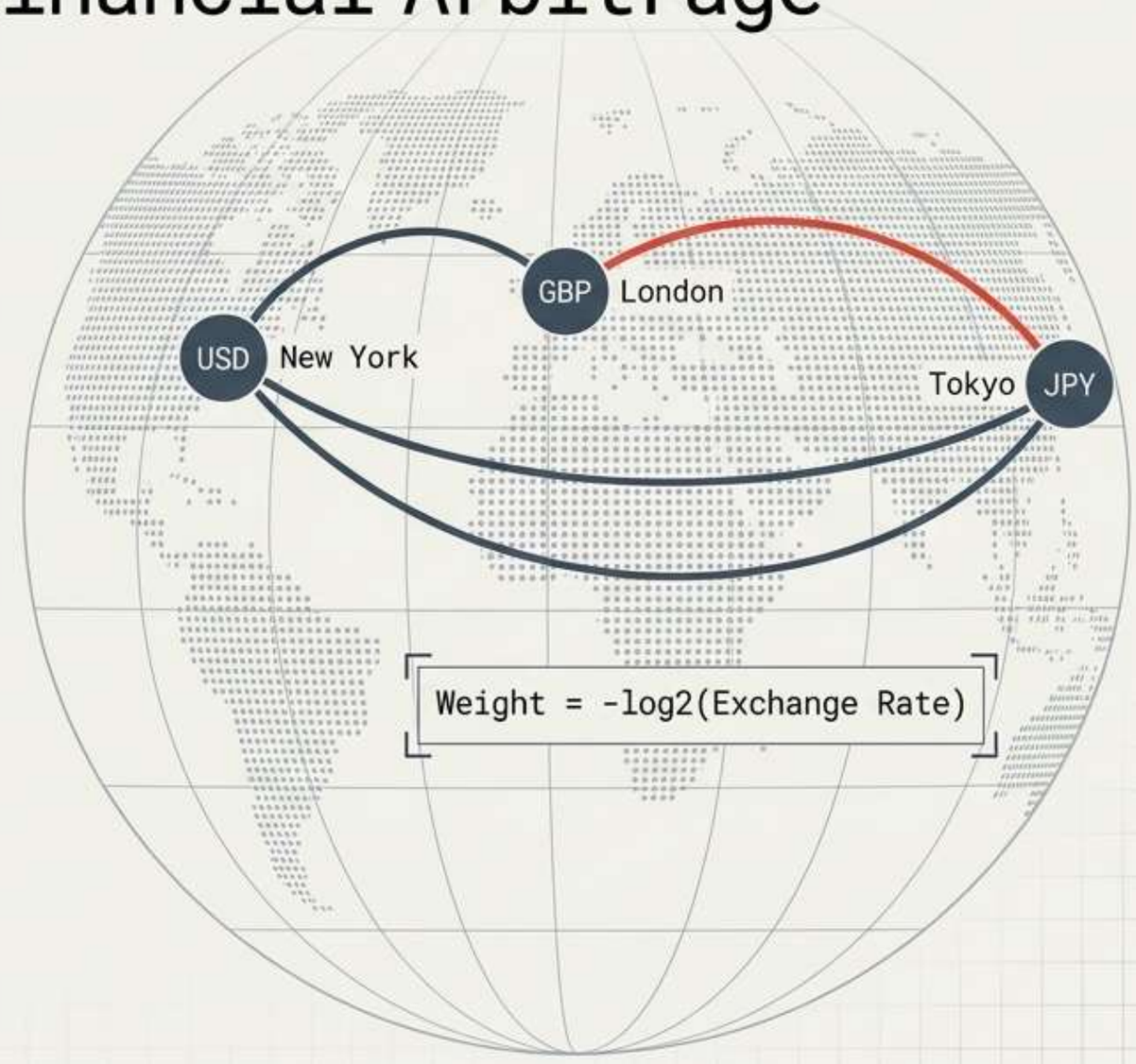
If the graph has strict directionality and absolutely no cycles (a **Directed Acyclic Graph**), Bellman-Ford is unnecessary. By processing vertices in Topological Sort order, shortest paths with negative weights can be found in blistering O(V+E) time.



Gemini Notebook

Real-World Execution: Financial Arbitrage

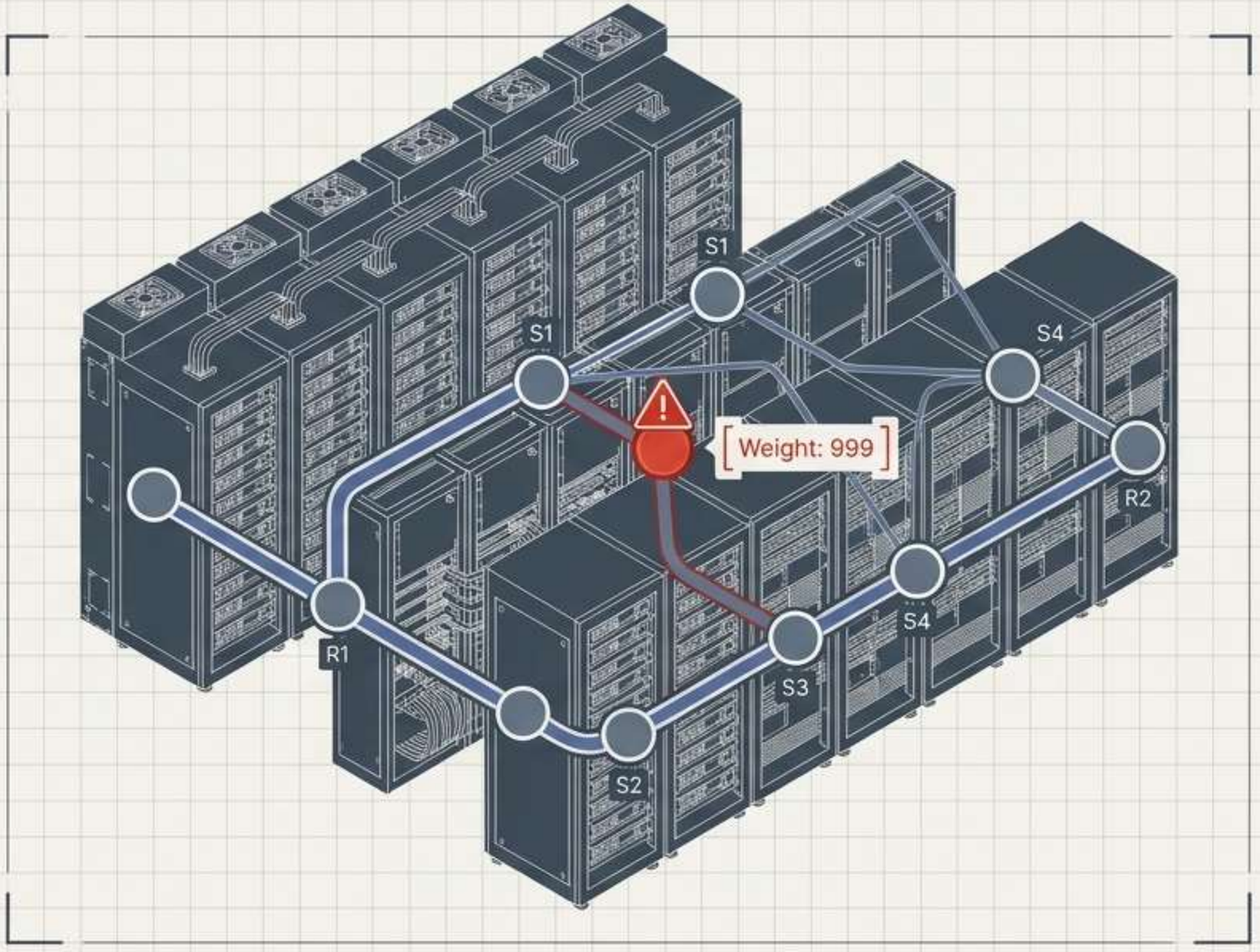
Quantitative trading firms use Bellman-Ford to hunt for risk-free profit loops. By taking the negative logarithm of foreign exchange rates and assigning them as edge weights, an arbitrage opportunity opportunity translates mathematically into a negative weight cycle. Phase 2 detects the cycle, signaling trade execution.



Gemini Notebook

Real-World Execution: Network Routing Protocols

Bellman-Ford forms the mathematical foundation of distance-vector routing protocols, such as `**RIP**`. It allows distributed computer networks to continuously continuously calculate the most efficient paths for data packets, dynamically updating as individual link costs fluctuate or hardware goes offline.



Gemini Notebook

The Definitive Pathfinding Matrix

	Dijkstra's Algorithm	**Bellman-Ford Algorithm**
Paradigm	Greedy Search	Dynamic Programming
Negative Edge Weights	✗ Fails	✓ Succeeds
Negative Cycle Detection	✗ Infinite Loop	✓ Detects & Flags
Time Complexity	$O(E \log V)$	$O(V \times E)$
Primary Use Case	Real-time GPS / Maps	Arbitrage & Network Routing

Gemini Notebook

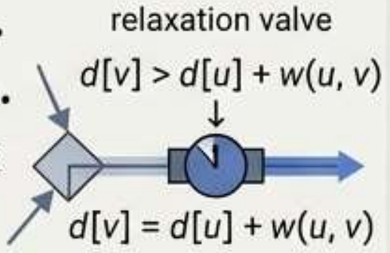
The Bellman-Ford Cheat Sheet

The What

SSSP (Single-Source Shortest Path) algorithm. Handles negative weights via Dynamic Programming.

The How

1. Initialize distances to ∞.
2. Relax all edges $V-1$ times.
3. Relax a V-th time to check for negative cycles.



The Complexity

Time: $O(V \times E)$

Space: $O(V)$

The Golden Rule

Use Dijkstra for speed on positive weights.

Use Bellman-Ford when network costs can be negative or routing requires safety.

Gemini Notebook